

Learning Mechanics

Introduction

The process of learning is quite similar to the evolution of a dynamical system.

Our work is not to solve a known dynamical system, but to find the laws that describe the learning process.

Deep learning theory should not be confined solely to Statistical Mechanics; it also requires Analytical Mechanics. Statistical Mechanics explains the macroscopic laws of neural networks, while Analytical Mechanics explains how parameters evolve under the drive of optimizer. The former studies information, energy, and scaling laws; the latter studies parameters, velocity, mass, and damping. In fact, the learning process of deep neural networks takes the coefficient vector as its object of study, with the network's infrastructure serving as constraints and the optimizer acting as the driving force that continuously updates the parameters. At its core, this is a continually evolving dynamical system. Therefore, we can study the learning process of deep neural networks using the methods of dynamical systems. A truly complete Scientific Theory of Deep Learning should, like the multi-scale theory in physics, be composed of both microscopic dynamics and macroscopic statistical laws.

The discrete numerical analysis adopted in this work employs implicit schemes, which naturally satisfy the description of dynamical systems via the discrete variational method. However, in engineering practice, explicit schemes are often preferred due to considerations of computational cost and efficiency. Implicit structures possess strong theoretical significance and can naturally yield the system's dynamical characteristics and macroscopic properties.

Furthermore, our framework provides the conditions under which the explicit approximation is valid. This constitutes a form of engineering value. When applying this theoretical framework to engineering practice, one must take into account the specific conditions to determine whether this theory can serve as a suitable approximation.

Definition for basic quantities

1. parameter state:

$$\theta_k$$

This quantity represents the "position" of the whole model in the parameter space

2. learning velocity:

$$v_k = \theta_k - \theta_{k-1}$$

This quantity represents the "velocity" of the learning machine

3.learning acceleration:

$$a_k = v_{k+1} - v_k = \theta_{k+1} - 2\theta_k + \theta_{k-1}$$

This quantity represents the rate of change of the learning velocity

Mathematical Foundation

Parameter Space

Function Space

Constraint Manifold

Physics Foundation : Action and Lagrange

The Lagrange of the learning process is:

$$\mathcal{L}(\theta_{k-1}, \theta_k) = \frac{1}{2}m_k \|\theta_k - \theta_{k-1}\|^2 - L(\theta_k)$$

L is loss function

So the discrete action is:

$$S_d = \sum_{k=1}^N \mathcal{L}(\theta_{k-1}, \theta_k) = \sum_{k=1}^N \left[\frac{1}{2}m_k \|\theta_k - \theta_{k-1}\|^2 - L(\theta_k) \right]$$

d'Alembert principle:

$$\begin{aligned} \delta S_d &= 0 \\ \Rightarrow D_2 \mathcal{L}(\theta_{k-1}, \theta_k) + D_1 \mathcal{L}(\theta_k, \theta_{k+1}) &= 0 \end{aligned}$$

The gradient of loss function:

$$g_k = \nabla_{\theta} L(\theta_k)$$

We assume that in the learning process, m is changing. And m is the function of the learning rate and gradient of loss function:

$$m_k = m(\eta_k, g_k)$$

Substitute and we can get:

$$\begin{aligned} D_2 \mathcal{L}(\theta_{k-1}, \theta_k) &= m_k(\theta_k - \theta_{k-1}) - g_k \\ D_1 \mathcal{L}(\theta_k, \theta_{k+1}) &= -m_{k+1}(\theta_{k+1} - \theta_k) \\ \Rightarrow m_{k+1}v_{k+1} - m_k v_k &= -g_k \end{aligned}$$

$$\Rightarrow v_{k+1} = \frac{m_k}{m_{k+1}} v_k - \frac{1}{m_{k+1}} g_k$$

We can write in the form of a and v:

$$m_{k+1} a_k + (m_{k+1} - m_k) v_k = -g_k$$

Adding dissipation into our equation:

$$m_{k+1} a_k + (m_{k+1} - m_k + \gamma_k) v_k = -g_k$$

In fact, if the learning rate doesn't change, the equation is:

$$\begin{aligned} m v_k &= (m - \gamma) v_{k-1} - g_k \\ \Rightarrow v_k &= \frac{m - \gamma}{m} v_{k-1} - \frac{1}{m} g_k \end{aligned}$$

update parameter:

$$\theta_k - \theta_{k-1} = v_k$$

Put optimizer into our framework

SGD

1. The condition for Gradient Descent

Standard Gradient Descent (without momentum) is characterized by:

$$v_{k+1} = -\eta g_k$$

This means the velocity does not depend on the previous velocity v_k . In terms of our momentum parameter β , this requires:

$$\beta = 1 - \frac{\gamma}{m} = 0$$

Hence:

$$\gamma = m$$

2. Substituting $\gamma = m$ into the velocity update

With $\gamma = m$, the velocity update becomes:

$$v_{k+1} = \left(1 - \frac{m}{m}\right) v_k - \frac{1}{m} g_k = 0 \cdot v_k - \frac{1}{m} g_k$$

Thus:

$$v_{k+1} = -\frac{1}{m} g_k$$

3. Parameter update

Using the parameter update rule $\theta_{k+1} = \theta_k + v_{k+1}$, we obtain:

$$\theta_{k+1} = \theta_k - \frac{1}{m} g_k$$

By defining the learning rate $\eta = \frac{1}{m}$, we recover the standard Gradient Descent update:

$$\theta_{k+1} = \theta_k - \eta g_k$$

Momentum SGD

$$v_{k+1} = \beta v_k - \eta g_k$$

$$\theta_{k+1} = \theta_k + v_{k+1}$$

1. Starting from the general equation

The learning dynamics equation with dissipation given in the document is:

$$m_{k+1} a_{k+1} + (m_{k+1} - m_k + \gamma_k) v_k = -g_k$$

2. Assumption of constant mass and dissipation coefficient

We assume:

$$m_k = m \quad (\text{constant}), \quad \gamma_k = \gamma \quad (\text{constant})$$

The general equation simplifies to:

$$m a_{k+1} + \gamma v_k = -g_k$$

Substituting

$$a_{k+1} = v_{k+1} - v_k$$

$$\Rightarrow m(v_{k+1} - v_k) + \gamma v_k = -g_k$$

Rearranging:

$$m v_{k+1} - m v_k + \gamma v_k = -g_k$$

$$\Rightarrow m v_{k+1} = (m - \gamma) v_k - g_k$$

Dividing both sides by

$$m$$

$$v_{k+1} = \left(1 - \frac{\gamma}{m}\right) v_k - \frac{1}{m} g_k$$

3. Correspondence with momentum SGD

The standard momentum SGD update rule is:

$$v_{k+1} = \beta v_k - \eta g_k$$

$$\theta_{k+1} = \theta_k + v_{k+1}$$

Comparing with our derived equation:

$$v_{k+1} = \underbrace{\left(1 - \frac{\gamma}{m}\right)}_{\beta} v_k - \underbrace{\frac{1}{m}}_{\eta} g_k$$

We obtain the parameter mapping:

$$\beta = 1 - \frac{\gamma}{m}, \quad \eta = \frac{1}{m}$$

Conversely, the physical parameters can be expressed in terms of momentum parameters:

$$m = \frac{1}{\eta}, \quad \gamma = \frac{1 - \beta}{\eta}$$

Adam

1. Variable-Mass Dissipative System Equation

From the theory section, the learning process satisfies:

$$m_{k+1} v_{k+1} = (m_k - \gamma_k) v_k - g_k$$

or equivalently:

$$v_{k+1} = \frac{m_k - \gamma_k}{m_{k+1}} v_k - \frac{1}{m_{k+1}} g_k$$

where

$$v_k = \theta_k - \theta_{k-1}$$

is the learning velocity and

$$g_k = \nabla L(\theta_k)$$

is the gradient of the loss function.

2. Adam Optimizer Update Rules

The standard form of Adam (with gradient descent direction) can be written as:

$$\begin{aligned}
v_{k+1} &= \beta_1 v_k - \eta_{\text{eff}}(1 - \beta_1)g_k, \\
\eta_{k+1} &= \beta_2 \eta_k + (1 - \beta_2)g_k^2, \\
\eta_{\text{eff}} &= \frac{\eta_0}{\sqrt{\eta_{k+1} + \varepsilon}}, \\
\theta_{k+1} &= \theta_k + v_{k+1}.
\end{aligned}$$

Here

$$v_k$$

is the parameter update (i.e., the velocity in the theory), and

$$\eta_{\text{eff}}$$

is the adaptive learning rate.

3. Coefficient Comparison

Comparing Adam's update equation with the theoretical equation:

$$v_{k+1} = \beta_1 v_k - \eta_{\text{eff}}(1 - \beta_1)g_k \quad \Longleftrightarrow \quad v_{k+1} = \frac{m_k - \gamma_k}{m_{k+1}}v_k - \frac{1}{m_{k+1}}g_k.$$

Equating the coefficients gives:

$$\frac{m_k - \gamma_k}{m_{k+1}} = \beta_1, \quad \frac{1}{m_{k+1}} = \eta_{\text{eff}}(1 - \beta_1).$$

From these, we obtain:

$$m_{k+1} = \frac{1}{\eta_{\text{eff}}(1 - \beta_1)}, \quad m_k = \frac{1}{\eta_{\text{eff}}^{(k-1)}(1 - \beta_1)},$$

where

$$\eta_{\text{eff}}^{(k-1)}$$

denotes the adaptive learning rate at the previous step. Then:

$$\gamma_k = m_k - \beta_1 m_{k+1} = \frac{1}{\eta_{\text{eff}}^{(k-1)}(1 - \beta_1)} - \frac{\beta_1}{\eta_{\text{eff}}^{(k)}(1 - \beta_1)} = \frac{1}{1 - \beta_1} \left(\frac{1}{\eta_{\text{eff}}^{(k-1)}} - \frac{\beta_1}{\eta_{\text{eff}}^{(k)}} \right).$$

4. Final Expressions

$$m_{k+1} = \frac{1}{\eta_{\text{eff}}^{(k)}(1 - \beta_1)}, \quad \gamma_k = \frac{1}{1 - \beta_1} \left(\frac{1}{\eta_{\text{eff}}^{(k-1)}} - \frac{\beta_1}{\eta_{\text{eff}}^{(k)}} \right)$$

where

$$\eta_{\text{eff}}^{(k)} = \frac{\eta_0}{\sqrt{\eta_{k+1} + \varepsilon}}$$

and

$$\eta_{k+1} = \beta_2 \eta_k + (1 - \beta_2) g_k^2$$

SGLD

The previous analyses treat the gradient g_k as exact. In large-scale learning, we typically use a mini-batch estimate $\tilde{g}_k$, which introduces gradient noise:

$$\tilde{g}_k = g_k + \zeta_k, \quad \mathbb{E}[\zeta_k] = 0, \quad \text{Cov}(\zeta_k) = \Sigma(\theta_k).$$

To capture the statistical properties of the learning process—especially for Bayesian posterior sampling—we must extend the dissipative dynamics with a Langevin thermal noise. The complete discrete stochastic dynamics becomes:

$$m_{k+1} v_{k+1} = (m_k - \gamma_k) v_k - \tilde{g}_k + \sqrt{2\gamma_k} \eta_k, \quad \eta_k \sim \mathcal{N}(0, I).$$

Here γ_k plays a dual role: it is both the dissipation coefficient that damps the momentum and the scale that sets the injected noise intensity. This exact correspondence is required by the fluctuation-dissipation theorem and guarantees that the stationary distribution of the system, when the learning rate is sufficiently small, is the Boltzmann distribution $p(\theta) \propto e^{-L(\theta)}$.

SGLD as an Overdamped Limit

Stochastic Gradient Langevin Dynamics (SGLD) corresponds to the case where all inertial terms vanish, i.e. the system is purely overdamped. From the momentum recurrence,

$$v_{k+1} = \frac{m_k - \gamma_k}{m_{k+1}} v_k - \frac{1}{m_{k+1}} \tilde{g}_k + \frac{\sqrt{2\gamma_k}}{m_{k+1}} \eta_k,$$

we see that momentum disappears when

$$\frac{m_k - \gamma_k}{m_{k+1}} = 0 \quad \Rightarrow \quad \gamma_k = m_k.$$

Assuming constant parameters $m_k = m$, $\gamma_k = \gamma = m$, the velocity update reduces to

$$v_{k+1} = -\frac{1}{m} \tilde{g}_k + \frac{\sqrt{2m}}{m} \eta_k = -\frac{1}{m} \tilde{g}_k + \sqrt{\frac{2}{m}} \eta_k.$$

If we define the learning rate $\epsilon = 1/m$, we directly obtain the standard SGLD update:

$$v_{k+1} = -\epsilon \tilde{g}_k + \sqrt{2\epsilon} \eta_k, \quad \Longleftrightarrow \quad \theta_{k+1} = \theta_k - \epsilon \tilde{g}_k + \sqrt{2\epsilon} \eta_k.$$

Thus, in the analytical mechanics framework, SGLD is characterized by:

- **Mass:** $m = 1/\epsilon$,
- **Dissipation:** $\gamma = m$ (exact balancing of friction and injection),
- **Noise:** naturally fixed to $\sqrt{2\gamma}$ by the fluctuation-dissipation relation.

Physical Interpretation and Significance

- **Overdamped motion:** The choice $\gamma = m$ means the system has no memory of its previous velocity; every step is determined solely by the current noisy gradient and the thermal noise. This is the deep-learning analogue of a particle moving in a highly viscous fluid under the influence of a potential $L(\theta)$.
- **Fluctuation-dissipation consistency:** Because the same γ governs both the dissipation of momentum and the variance of the injected noise, the dynamics automatically relax to the desired equilibrium distribution. Any mismatch would create an effective temperature different from unity and corrupt the Bayesian posterior.
- **Separation of gradient noise and thermal noise:** The gradient noise ζ_k enters the update multiplied by ϵ , while the thermal noise enters with strength $\sqrt{2\epsilon}$. In the limit $\epsilon \rightarrow 0$, the gradient noise becomes negligibly small compared to the thermal noise. The dynamics thus approximate the continuous-time overdamped Langevin equation with exact gradient g_k , whose stationary measure is $e^{-L(\theta)}$. SGLD therefore acts as a valid posterior sampler even when using mini-batch gradients.
- **Unification of microscopic dynamics and macroscopic statistics:** In this framework, the variational principle provides the conservative skeleton (m, v, a) , the dissipation γ breaks time-reversal symmetry, and the Langevin noise restores thermal equilibrium. The parameters that define the optimizer's trajectory are exactly those that guarantee the correct macroscopic statistical behavior—this is a direct realization of the multi-scale vision stated in the introduction.

Function-Space Dynamics: From Parameter Kinetics to the Neural Tangent Kernel

1. From Parameter Dynamics to Function Dynamics

We begin with the discrete variational principle introduced earlier. For constant mass m and dissipation γ , the parameter update rule (discrete Newton equation) reads

$$mv_{k+1} = (m - \gamma)v_k - g_k,$$

where $v_k = \theta_k - \theta_{k-1}$ is the parameter velocity and $g_k = \nabla_{\theta} L(\theta_k)$ is the loss gradient.

Define the **function state vector** on the training set $\{x_i\}_{i=1}^N$ as

$$\mathbf{F}_k = [f(\theta_k, x_1), f(\theta_k, x_2), \dots, f(\theta_k, x_N)]^\top.$$

The function velocity and acceleration are the discrete differences

$$\mathbf{V}_k = \mathbf{F}_k - \mathbf{F}_{k-1}, \quad \mathbf{A}_k = \mathbf{V}_{k+1} - \mathbf{V}_k = \mathbf{F}_{k+1} - 2\mathbf{F}_k + \mathbf{F}_{k-1}.$$

Let $\mathbf{J}_k = \frac{\partial \mathbf{F}}{\partial \theta} \big|_{\theta_k} \in \mathbb{R}^{N \times P}$ be the Jacobian of the network outputs with respect to the parameters.

For small parameter changes we have the local linear approximation

$$\mathbf{V}_k \approx \mathbf{J}_k v_k.$$

To obtain the function-space evolution equation, we evaluate $\mathbf{A}_k$ using the parameter dynamics:

$$\begin{aligned} \mathbf{A}_k &= \mathbf{J}_{k+1} v_{k+1} - \mathbf{J}_k v_k \\ &= \mathbf{J}_{k+1} \left(\frac{m - \gamma}{m} v_k - \frac{1}{m} g_k \right) - \mathbf{J}_k v_k \end{aligned}$$

For constant m and γ , this simplifies to

$$\boxed{\mathbf{A}_k + \frac{\gamma}{m} \mathbf{V}_k + \frac{1}{m} \mathbf{J}_k g_k = \Delta \mathbf{J}_k v_k} \quad (1)$$

where $\Delta \mathbf{J}_k = \mathbf{J}_{k+1} - \mathbf{J}_k$.

Equation (1) is the **exact discrete function-dynamics equation** for finite-width networks. It contains:

- an inertial term $\mathbf{A}_k$,
- a dissipation term $\frac{\gamma}{m} \mathbf{V}_k$,
- a restoring force $\frac{1}{m} \mathbf{J}_k g_k$,
- and a geometric source term $\Delta \mathbf{J}_k v_k$ due to the evolution of the Jacobian.

2. Emergence of the Neural Tangent Kernel

For the mean-squared error loss,

$$L(\theta) = \frac{1}{2} \|\mathbf{F}(\theta) - \mathbf{y}\|^2,$$

the gradient is $g_k = \mathbf{J}_k^\top (\mathbf{F}_k - \mathbf{y})$. Inserting this into (1) yields

$$\mathbf{A}_k + \frac{\gamma}{m} \mathbf{V}_k + \frac{1}{m} \mathbf{J}_k \mathbf{J}_k^\top (\mathbf{F}_k - \mathbf{y}) = \Delta \mathbf{J}_k v_k.$$

Define the **neural tangent kernel (NTK) matrix** at step k :

$$\mathbf{K}_k = \mathbf{J}_k \mathbf{J}_k^\top \in \mathbb{R}^{N \times N}$$

Its entries are the pairwise inner products of the parameter gradients:

$$(\mathbf{K}_k)_{ij} = \langle \nabla_{\theta} f(\theta_k, x_i), \nabla_{\theta} f(\theta_k, x_j) \rangle.$$

Thus the function-dynamics equation becomes

$$\mathbf{A}_k + \frac{\gamma}{m} \mathbf{V}_k + \frac{1}{m} \mathbf{K}_k (\mathbf{F}_k - \mathbf{y}) = \Delta \mathbf{J}_k v_k \quad (2)$$

The NTK matrix appears naturally as the **elastic stiffness tensor** in function space: a deviation $(\mathbf{F}_k - \mathbf{y})$ creates a restoring force $-\frac{1}{m} \mathbf{K}_k (\mathbf{F}_k - \mathbf{y})$.

3. The NTK Regime: Frozen Kernel and Lazy Training

The standard NTK theory corresponds to the limit $m \rightarrow \infty$ (the “lazy training” regime). In this limit:

- Parameter velocity $v_k \rightarrow 0$, therefore $\theta_k \approx \theta_0$ for all k .
- The Jacobian becomes frozen: $\mathbf{J}_k \approx \mathbf{J}_0$, $\Delta \mathbf{J}_k \approx 0$.
- The kernel matrix becomes constant: $\mathbf{K}_k \approx \mathbf{K}_0 = \mathbf{J}_0 \mathbf{J}_0^\top$.
- The function acceleration $\mathbf{A}_k$ and the source term $\Delta \mathbf{J}_k v_k$ become negligible.

Under these conditions equation (2) reduces to

$$\frac{\gamma}{m} \mathbf{V}_k + \frac{1}{m} \mathbf{K}_0 (\mathbf{F}_k - \mathbf{y}) = 0.$$

Substituting $\mathbf{V}_k = \mathbf{F}_k - \mathbf{F}_{k-1}$ we obtain the **implicit linear NTK dynamics**:

$$\mathbf{F}_k - \mathbf{F}_{k-1} = -\frac{1}{m} \mathbf{K}_0 (\mathbf{F}_k - \mathbf{y}) \quad (3)$$

For standard gradient descent (SGD without momentum) we have $\gamma = m$ and learning rate $\eta = 1/m$, so

$$\mathbf{F}_k - \mathbf{F}_{k-1} = -\eta \mathbf{K}_0 (\mathbf{F}_k - \mathbf{y}).$$

This is an implicit (backward Euler) scheme: the update depends on the current $\mathbf{F}_k$. Because $\mathbf{K}_0$ is fixed, the system is **linear and autonomous**.

4. Explicit Approximation and Its Engineering Relevance

In practice, deep learning implementations use an **explicit** (forward Euler) update. The implicit equation can be rewritten as

$$\mathbf{F}_k = (\mathbf{I} + \eta \mathbf{K}_0)^{-1} (\mathbf{F}_{k-1} + \eta \mathbf{K}_0 \mathbf{y}).$$

Expanding the matrix inverse in a Neumann series,

$$(\mathbf{I} + \eta \mathbf{K}_0)^{-1} = \mathbf{I} - \eta \mathbf{K}_0 + \eta^2 \mathbf{K}_0^2 - \dots,$$

and truncating after the first order yields the **explicit NTK dynamics**

$$\boxed{\mathbf{F}_k - \mathbf{F}_{k-1} = -\eta \mathbf{K}_0 (\mathbf{F}_{k-1} - \mathbf{y})} \quad (4)$$

The truncation is valid when the spectral norm of $\eta \mathbf{K}_0$ is strictly less than one:

$$\boxed{\eta \|\mathbf{K}_0\| < 1}$$

This condition guarantees:

1. **Stability**: The linear iteration matrix $\mathbf{I} - \eta \mathbf{K}_0$ has all eigenvalues in $(0, 1]$, ensuring monotone convergence without oscillations.
2. **Fidelity**: The Neumann series converges, so the explicit scheme faithfully approximates the implicit dynamics with an error of order $\mathcal{O}(\eta^2 \mathbf{K}_0^2)$.

In our theoretical framework, this condition is naturally satisfied in the **NTK phase** where the learning Reynolds number $R \ll 1$ and the kernel is frozen. During active feature learning ($R \sim 1$), the kernel $\mathbf{K}_k$ evolves and $\|\mathbf{K}_k\|$ may increase, requiring dynamic monitoring of $\eta \|\mathbf{K}_k\|$. This provides a theoretical underpinning for learning-rate warmup and decay strategies used in practice.

5. Summary: From Parameters to Functions

Level	Discrete Equation	Description
Parameter dynamics	$mv_{k+1} = (m - \gamma)v_k - g_k$	Newton's second law in θ -space
Jacobian mapping	$\mathbf{V}_k \approx \mathbf{J}_k v_k$	Projects parameter velocities to function space
Full function dynamics	$\mathbf{A}_k + \frac{\gamma}{m} \mathbf{V}_k + \frac{1}{m} \mathbf{K}_k (\mathbf{F}_k - \mathbf{y}) = \Delta \mathbf{J}_k v_k$	Exact discrete dynamics (finite width)
NTK (lazy) limit	$\mathbf{F}_k - \mathbf{F}_{k-1} = -\eta \mathbf{K}_0 (\mathbf{F}_k - \mathbf{y})$	Implicit linear dynamics, frozen kernel

Level	Discrete Equation	Description
Explicit (engineering) form	$\mathbf{F}_k - \mathbf{F}_{k-1} = -\eta \mathbf{K}_0(\mathbf{F}_{k-1} - \mathbf{y})$	Valid when $\eta \ \mathbf{K}_0\ < 1$

The function-space perspective unifies the NTK theory (as the $m \rightarrow \infty$, $R \ll 1$ limit) with the richer, finite-width feature-learning regime where the kernel evolves. The discrete variational principle provides a single, self-consistent mathematical language for both worlds.

From Discrete Optimization to Continuous Gradient Flow: The Three Necessary Conditions

1. Background and Motivation

In deep learning theory, the learning rate η plays a role analogous to Planck's constant $\hbar$ in quantum mechanics. Just as quantum mechanics reduces to classical mechanics when $\hbar \rightarrow 0$, discrete stochastic gradient descent (SGD) algorithms reduce to continuous-time gradient flows when $\eta \rightarrow 0$.

However, the critical question remains: **How small is "small enough" for the learning rate?**

According to the discrete variational principle outlined in the document, the discrete action S_d converges to the continuous Lagrangian mechanics under certain conditions. This convergence is not merely asymptotic; it relies strictly on three mathematical inequalities.

2. The Three Necessary Conditions

To ensure that the discrete iterative updates accurately approximate a continuous dynamical system, the following three conditions must be simultaneously satisfied.

Condition 1: Smoothness of the Loss Landscape (Hessian Constraint)

The loss function $L(\theta)$ must change smoothly within a single step η . This requires that the local curvature of the loss landscape is bounded.

$$\eta \|\nabla^2 L(\theta_k)\| \ll 1$$

Theoretical Origin: This is derived from the first-order Taylor expansion of the gradient. If this condition is violated, the gradient changes too abruptly within a single step, rendering the explicit Euler approximation unstable. In the context, this directly correlates with the stability condition $\eta \|\mathbf{K}_0\| < 1$ for the frozen NTK regime. It ensures that the local linear approximation of the gradient is valid.

Condition 2: Trajectory Curvature (Geometric Constraint)

The curvature radius ρ of the parameter trajectory in phase space must be significantly larger than the step size.

$$\rho \gg |\theta_k - \theta_{k-1}| \sim \eta \|\nabla L(\theta_k)\|$$

Theoretical Origin: This condition ensures the parameter path does not make "sharp turns" in a single iteration. In numerical analysis, a sharp turn indicates that the acceleration (the second derivative of parameters) is unbounded within one discrete step. This violates the premise of continuous Newtonian mechanics, where velocity changes are infinitesimal. Only when this holds can we treat the trajectory as a smooth continuous curve in the parameter manifold.

Condition 3: Frozen Kernel Dynamics (Feature-Learning Constraint)

For function-space dynamics, the Neural Tangent Kernel (NTK) must evolve slowly enough that its change within one step is negligible.

Based on the discrete function-dynamics equation derived in *Theory.pdf* (Page 7, Equation 1):

$$\mathbf{A}_k + \frac{\gamma}{m} \mathbf{V}_k + \frac{1}{m} \mathbf{K}_k (\mathbf{F}_k - \mathbf{y}) = \Delta \mathbf{J}_k v_k$$

The geometric source term $\Delta \mathbf{J}_k v_k$ must vanish. Using the mean value theorem, $\Delta \mathbf{J}_k \approx \nabla_{\theta} \mathbf{J} \cdot (\theta_{k+1} - \theta_k) \sim \eta \nabla_{\theta} \mathbf{J} \cdot \nabla L$. Thus, the strict condition is:

$$\eta \|\nabla_{\theta} \mathbf{J}(\theta_k)\| \|\nabla L(\theta_k)\| \ll \|\mathbf{J}(\theta_k)\|$$

Theoretical Origin: This is the exact mathematical foundation of the **Lazy Training (NTK) regime**. If this condition fails, the kernel $\mathbf{K}_k$ becomes dynamic, and the network enters the "active feature learning" phase ($R \sim 1$). In that case, the continuous limit of the function dynamics is no longer a linear constant-coefficient differential equation, and the discrete variational principle must be retained.

3. The Unified Bound for the Learning Rate

Combining the three conditions, the learning rate η must satisfy the following **strict upper bound** to ensure the validity of the continuous limit:

$$\eta \ll \min \left\{ \frac{1}{\|\nabla^2 L\|}, \frac{\rho}{\|\nabla L\|}, \frac{\|\mathbf{J}\|}{\|\nabla_{\theta} \mathbf{J}\| \|\nabla L\|} \right\}$$

4. Engineering Significance

In practical deep learning, we do not explicitly calculate these supremums. However, strategies like **Learning Rate Warm-up** and **Decay** are physical manifestations of these constraints:

- **Warm-up** forces the system to start in the continuous gradient-flow regime (where $R \ll 1$) to avoid early explosive chaos.
- **Decay** ensures that the system remains within the valid continuous region as the Jacobian norm $|\mathbf{J}|$ and the Hessian curvature potentially increase during training.

Conclusion: If η violates any of these three inequalities, the discrete effects become prominent. The continuous calculus of variations cannot adequately describe the learning process. In such cases, one must resort to the **discrete variational method** ($\delta S_d = 0$) introduced in [Theory.pdf](#), which naturally accommodates the discrete nature of explicit optimizers and remains mathematically self-consistent even in the "quantum" (large η) regime.

Learning Fluid Mechanics: Describe Macroscopic Behavior

Inertia-Dissipation Ratio/Learning Reynolds Number

Reynolds Number

$$R = \frac{\text{Inertia}}{\text{Dissipation}} = \frac{|m_k \dot{v}|}{|\gamma_k v|}$$

Learning Process = inertia - dominated region + transition region + gradient flow region

1. $R \gg 1$

$$m\dot{v} \approx -\nabla L$$

The parameter loss obvious memories

2. $R \sim 1$

$$m\dot{v} \approx \nabla L$$

The loss may decrease fastest in this region

3. $R \ll 1$

$$\gamma\dot{v} \approx -\nabla L$$

gradient flow

Deep Learning Generalization: A Detailed Derivation via Hamiltonian Mechanics

1. Construction of the Learning Hamiltonian

We treat the training dynamics in parameter space $\theta \in \mathbb{R}^P$ as a mechanical evolution.

Discrete Lagrangian:

$$L(\theta_{k-1}, \theta_k) = \frac{1}{2} m_k \|\theta_k - \theta_{k-1}\|^2 - \mathcal{L}(\theta_k),$$

where $v_k = \theta_k - \theta_{k-1}$ is the step velocity.

Momentum definition:

$$p_k \triangleq \frac{\partial L}{\partial v_k} = m_k v_k = m_k (\theta_k - \theta_{k-1}).$$

Hamiltonian via Legendre transformation:

$$H(\theta_k, p_k) = p_k v_k - L(\theta_{k-1}, \theta_k).$$

Substituting $v_k = p_k / m_k$:

$$H = p_k \left(\frac{p_k}{m_k} \right) - \left[\frac{1}{2} m_k \left(\frac{p_k}{m_k} \right)^2 - \mathcal{L}(\theta_k) \right] = \frac{|p_k|^2}{m_k} - \frac{|p_k|^2}{2m_k} + \mathcal{L}(\theta_k) = \frac{|p_k|^2}{2m_k} + \mathcal{L}(\theta_k).$$

Thus,

$$H(\theta_k, p_k) = T_k + U(\theta_k), \quad T_k = \frac{|p_k|^2}{2m_k}, \quad U(\theta_k) = \mathcal{L}(\theta_k).$$

The kinetic energy T_k and potential energy (loss) $U(\theta_k)$ are identified.

2. Exact Jacobian in Phase Space

The discrete update rule with damping γ maps $\Phi_k : (\theta_k, v_k) \rightarrow (\theta_{k+1}, v_{k+1})$:

$$\begin{cases} v_{k+1} = \frac{m_k - \gamma_k}{m_{k+1}} v_k - \frac{1}{m_{k+1}} g_k, \\ \theta_{k+1} = \theta_k + v_{k+1}, \end{cases}$$

where $g_k = \nabla_{\theta} \mathcal{L}(\theta_k)$.

Define the Hessian $H_k = \nabla_{\theta}^2 \mathcal{L}(\theta_k)$. The block Jacobian is

$$D\Phi_k = \begin{bmatrix} \frac{\partial \theta_{k+1}}{\partial \theta_k} & \frac{\partial \theta_{k+1}}{\partial v_k} \\ \frac{\partial v_{k+1}}{\partial \theta_k} & \frac{\partial v_{k+1}}{\partial v_k} \end{bmatrix} = \begin{bmatrix} I - A & B \\ -A & B \end{bmatrix},$$

with

$$A = \frac{1}{m_{k+1}} H_k, \quad B = \frac{m_k - \gamma_k}{m_{k+1}} I.$$

Determinant via Schur complement:

$$\det(D\Phi_k) = \det(I - A) \det(B - (-A)(I - A)^{-1}B).$$

Simplify the inner factor:

$$B - (-A)(I - A)^{-1}B = B + A(I - A)^{-1}B = [I + A(I - A)^{-1}]B.$$

Since

$$I + A(I - A)^{-1} = (I - A)(I - A)^{-1} + A(I - A)^{-1} = (I - A + A)(I - A)^{-1} = (I - A)^{-1},$$

we get

$$\det(D\Phi_k) = \det(I - A) \cdot \det((I - A)^{-1}B) = \det(B).$$

Therefore,

$$J_k = \det(D\Phi_k) = \left(\frac{m_k - \gamma_k}{m_{k+1}} \right)^P.$$

3. Statistical Mechanics: Entropy and Partition Function

3.1 Entropy Evolution

The phase-space volume transforms as $d\theta' dv' = J_k d\theta dv$, and probability density evolves as $P_{k+1} = P_k / J_k$. Then

$$S_{k+1} = - \int P_{k+1} \ln P_{k+1} d\Omega' = - \int \frac{P_k}{J_k} \ln \left(\frac{P_k}{J_k} \right) J_k d\Omega = - \int P_k (\ln P_k - \ln J_k) d\Omega.$$

Hence

$$S_{k+1} = S_k + \ln J_k \int P_k d\Omega = S_k + \ln J_k,$$

since $\int P_k d\Omega = 1$. Therefore,

$$S_{k+1} - S_k = \ln J_k = P \ln \left(\frac{m_k - \gamma_k}{m_{k+1}} \right).$$

3.2 Partition Function from Maximum Entropy Principle

Maximize entropy S subject to energy constraint $\langle H_k \rangle = E_k$ using Lagrange multipliers:

$$\mathcal{F}[P] = - \int P \ln P d\Omega - \alpha_k \left(\int P d\Omega - 1 \right) - \beta_k \left(\int P H_k d\Omega - E_k \right).$$

Variation w.r.t. P :

$$\frac{\delta \mathcal{F}}{\delta P} = -\ln P - 1 - \alpha_k - \beta_k H_k = 0 \implies P_k \propto \exp(-\beta_k H_k).$$

Normalisation gives the **partition function**:

$$Z_k = \int_{\mathbb{R}^{2P}} \exp(-\beta_k H_k) d\theta dv = \left(\int e^{-\beta_k \mathcal{L}(\theta)} d\theta \right) \cdot \left(\int e^{-\frac{\beta_k}{2} m_k v^2} dv \right)^P.$$

The momentum integral is Gaussian:

$$\int e^{-\frac{\beta_k}{2} m_k v^2} dv = \sqrt{\frac{2\pi}{\beta_k m_k}}.$$

Thus,

$$Z_k = \left(\int e^{-\beta_k \mathcal{L}(\theta)} d\theta \right) \cdot \left(\sqrt{\frac{2\pi}{\beta_k m_k}} \right)^P.$$

4. Thermodynamic Decomposition of Free Energy

The free energy is defined as

$$F_k \triangleq E_k - T_k S_k = -\frac{1}{\beta_k} \ln Z_k,$$

where $T_k = 1/\beta_k$ (in units where $k_B = 1$).

Consider the change $\Delta F_k = F_{k+1} - F_k$. Let

$$E_{k+1} = E_k + \Delta E, \quad T_{k+1} = T_k + \Delta T, \quad S_{k+1} = S_k + \Delta S.$$

Then

$$\Delta F_k = (E_k + \Delta E) - (T_k + \Delta T)(S_k + \Delta S) - (E_k - T_k S_k) = \Delta E - [T_k \Delta S + S_k \Delta T + \Delta T \Delta S].$$

Since $S_{k+1} \Delta T = (S_k + \Delta S) \Delta T = S_k \Delta T + \Delta T \Delta S$, we can write

$$\Delta F_k = \Delta E_k - [T_k(\Delta S_k) + S_{k+1}(\Delta T_k)].$$

5. Bridge to Generalization Theory

5.1 PAC-Bayes Foundation

The PAC-Bayes theorem provides a theoretical connection between the training loss and the generalization error. For a posterior distribution P_k^θ over parameters and a fixed prior π , we have

$$\text{Gen}(P_k^\theta) \leq \langle \mathcal{L}_{\text{train}} \rangle_{P_k^\theta} + \frac{1}{\beta_k} \text{KL}(P_k^\theta \mid \pi).$$

Using the identity $\text{KL}(P_k^\theta \mid \pi) = -S_k^\theta + C_\pi$ (where S_k^θ is the entropy of the marginal distribution and C_π is a constant dependent on the prior), and recalling the definition of the free energy $F_k = \langle \mathcal{L} \rangle - T_k S_k$, we obtain

$$\boxed{\text{Gen}(P_k^\theta) \leq F_k + \frac{1}{\beta_k} C_\pi.}$$

Thus, the free energy F_k serves as a theoretical upper bound on the generalization error (up to an additive constant). This justifies using changes in free energy ΔF_k as a proxy for changes in the generalization bound.

5.2 Operational Flatness: Microscopic Definition

We define the **microscopic operational flatness** Ω_k at step k as the ratio of the change in free energy to the change in the average training loss:

$$\boxed{\Omega_k \triangleq \frac{F_{k+1} - F_k}{\langle \mathcal{L} \rangle_{k+1} - \langle \mathcal{L} \rangle_k} = \frac{\Delta F_k}{\Delta \langle \mathcal{L} \rangle_k} .}$$

Interpretation: Ω_k measures the amount of free energy (i.e., the generalisation upper bound) decreased per unit decrease in the training loss. It quantifies the *leverage* that a drop in training loss has on improving generalisation.

Threshold criteria for Ω_k :

Condition	Classification	Implication
$\Omega_k = 1$	Optimal flatness	The drop in free energy exactly matches the drop in loss. Every reduction in training loss translates into an equal reduction in the generalisation bound.
$0 < \Omega_k \leq 1$	Good flatness	The free energy decreases along with the loss, indicating healthy generalisation improvement (though suboptimal if < 1).
$\Omega_k = 0$	Neutral point	The loss decreases ($\Delta \langle \mathcal{L} \rangle < 0$), but the free energy remains unchanged ($\Delta F = 0$). Generalisation does not improve

Condition	Classification	Implication
		despite training progress; computational cost is wasted.
$\Omega_k < 0$	Sharp / Overfitting	The loss decreases ($\Delta\langle\mathcal{L}\rangle < 0$), but the free energy increases ($\Delta F > 0$). The generalisation bound worsens, indicating that the model is overfitting to the training data.

5.3 Operational Flatness: Macroscopic (Global) Definition

To assess the overall training efficiency over a long interval from step k_1 to step k_2 ($k_2 > k_1$), we define the **global operational flatness** as the ratio of the total decreases:

$$\bar{\Omega}[k_1, k_2] \triangleq \frac{F_{k_1} - F_{k_2}}{\langle\mathcal{L}\rangle_{k_1} - \langle\mathcal{L}\rangle_{k_2}}.$$

Here:

- $F_{k_1} - F_{k_2}$ is the total decrease in free energy over the interval.
- $\langle\mathcal{L}\rangle_{k_1} - \langle\mathcal{L}\rangle_{k_2}$ is the total decrease in training loss.

5.4 Weighted-Average Derivation of the Global Flatness

The global flatness can be expressed as a weighted average of the microscopic flatness values Ω_k . Let $\Delta\langle\mathcal{L}\rangle_k \triangleq \langle\mathcal{L}\rangle_{k+1} - \langle\mathcal{L}\rangle_k$. By definition, $\Omega_k = \Delta F_k / \Delta\langle\mathcal{L}\rangle_k$, which implies $\Delta F_k = \Omega_k \Delta\langle\mathcal{L}\rangle_k$.

Now, summing over the interval $[k_1, k_2]$:

$$\sum_{k=k_1}^{k_2-1} \Delta F_k = F_{k_2} - F_{k_1} = -(F_{k_1} - F_{k_2}),$$

and

$$\sum_{k=k_1}^{k_2-1} \Delta\langle\mathcal{L}\rangle_k = \langle\mathcal{L}\rangle_{k_2} - \langle\mathcal{L}\rangle_{k_1} = -(\langle\mathcal{L}\rangle_{k_1} - \langle\mathcal{L}\rangle_{k_2}).$$

Therefore, the global flatness becomes

$$\bar{\Omega}[k_1, k_2] = \frac{F_{k_1} - F_{k_2}}{\langle\mathcal{L}\rangle_{k_1} - \langle\mathcal{L}\rangle_{k_2}} = \frac{-(F_{k_2} - F_{k_1})}{-(\langle\mathcal{L}\rangle_{k_2} - \langle\mathcal{L}\rangle_{k_1})} = \frac{\sum_{k=k_1}^{k_2-1} \Delta F_k}{\sum_{k=k_1}^{k_2-1} \Delta\langle\mathcal{L}\rangle_k}.$$

Substituting $\Delta F_k = \Omega_k \Delta\langle\mathcal{L}\rangle_k$ yields the final weighted-average form:

$$\bar{\Omega}[k_1, k_2] = \frac{\sum_{k=k_1}^{k_2-1} \Omega_k \Delta\langle\mathcal{L}\rangle_k}{\sum_{k=k_1}^{k_2-1} \Delta\langle\mathcal{L}\rangle_k}.$$

Interpretation: The global flatness is the weighted average of the local flatness values, where each local Ω_k is weighted by the corresponding loss decrease $\Delta\langle\mathcal{L}\rangle_k$ (note that these weights are negative, as the loss decreases over training). This provides a holistic measure of how effectively the entire training trajectory converts loss reduction into generalisation improvement.